Abstract

La clase se centra en cómo diseñar y controlar agentes de inteligencia artificial capaces de razonar, utilizar herramientas y resolver tareas de forma autónoma. Se profundiza en dos mecanismos fundamentales: ReAct (Reasoning and Acting), que combina razonamiento y ejecución, y Tool Calling, que permite al modelo solicitar herramientas mediante llamadas estructuradas.

A partir de ejemplos de un asistente de atención al cliente para una tienda de comercio electrónico llamada Lumenshop, se analizan los riesgos de las llamadas repetitivas, las alucinaciones, los errores de ejecución y el uso excesivo de herramientas. También se estudian arquitecturas con varios agentes, como los sistemas de enrutamiento, planificación y ejecución, así como la importancia de evaluar resultados, controlar costes y establecer mecanismos de intervención humana.

La idea principal es que no basta con que un agente sea capaz de razonar: también hay que estructurar sus acciones, controlar su ejecución y verificar sus resultados.

Resumen general

ReAct: combinar razonamiento y acción

ReAct integra el razonamiento del modelo con la ejecución de acciones. El agente plantea qué necesita hacer, selecciona una herramienta, ejecuta la acción y analiza el resultado obtenido. Si la respuesta no es suficiente, puede ajustar su estrategia y continuar hasta resolver la tarea. Este enfoque permite conectar la planificación con los resultados reales de las herramientas, en lugar de limitarse a generar una secuencia de pensamientos.

Tool Calling: llamadas estructuradas a herramientas

El Tool Calling permite que un modelo solicite la ejecución de una herramienta mediante una llamada estructurada, en lugar de depender únicamente de texto libre. El código recibe esa solicitud, ejecuta la función y devuelve el resultado al modelo. Este proceso puede repetirse hasta que el agente tenga la información necesaria para responder. Según la clase, es una alternativa especialmente importante en entornos empresariales por el mayor control que permite sobre las acciones del sistema.

Este documento ha sido generado con inteligencia artificial.

Diferencias entre ReAct y Tool Calling

La diferencia central está en cómo se organiza la interacción entre el modelo y el código. En ReAct, el desarrollador define un formato para que el modelo describa sus pasos de razonamiento, acción, entrada y observación. En Tool Calling, el modelo genera directamente llamadas estructuradas que el código puede ejecutar. La elección depende de las capacidades del modelo y de los requisitos del proyecto; para producción empresarial, la clase destaca la utilidad del soporte nativo de Tool Calling.

Diseño y organización de las herramientas

Las herramientas deben tener nombres descriptivos, instrucciones claras y parámetros con formatos definidos. Esto ayuda al modelo a seleccionar la función adecuada y reduce errores en los datos de entrada. Cuando un agente tiene un catálogo demasiado grande de herramientas, se puede dividir la arquitectura en agentes especializados, por ejemplo, uno para pedidos, otro para atención al cliente y otro para reembolsos. Un agente enrutador recibe la solicitud y la dirige al especialista correspondiente, reduciendo el contexto y el coste de ejecución.

Prevención de bucles y condiciones de parada

Un agente puede repetir una acción indefinidamente si no encuentra un cliente, recibe datos incorrectos o no consigue completar una tarea. Para evitarlo, se pueden establecer límites de iteraciones, detectar llamadas repetidas y devolver mensajes que indiquen al modelo que debe detenerse o solicitar información adicional. La clase insiste en que este control debe implementarse desde el código, sin depender exclusivamente de que el modelo reconozca sus propios errores.

Control del formato y fiabilidad de las respuestas

Aunque las llamadas estructuradas mejoran el control, el desarrollador sigue siendo responsable de comprobar que el modelo respete el formato esperado. Se pueden definir esquemas de salida, utilizar JSON, validar los parámetros y establecer reintentos cuando la respuesta no cumple las condiciones. También es importante probar distintos modelos, porque su comportamiento puede variar ante una misma solicitud. La estructura ayuda a reducir errores, pero no elimina la necesidad de validación.

Planificación, ejecución y síntesis

Uno de los patrones estudiados separa la resolución de una tarea en tres funciones. Primero, un planificador descompone el objetivo en pasos; después, un ejecutor realiza esos pasos y recoge los resultados; finalmente, un sintetizador utiliza la información obtenida para construir la respuesta final. Esta separación permite inspeccionar el plan antes de ejecutarlo, identificar errores y facilitar que una persona revise o apruebe las acciones cuando sea necesario.

Este documento ha sido generado con inteligencia artificial.

Reflexión y replanteamiento del plan

La clase introduce la importancia de evaluar los resultados durante la ejecución, en lugar de asumir que el plan inicial siempre funcionará. Si un paso falla, el sistema puede analizar el problema y generar una alternativa. Por ejemplo, si un servicio externo no responde, no debería continuar como si la operación hubiera sido exitosa. El reto consiste en incorporar mecanismos de reflexión y replanteamiento que permitan recuperarse de los errores sin generar nuevos bucles ni ejecutar acciones innecesarias.

Evaluación de las estrategias de agentes

Para comparar diferentes arquitecturas, se propone ejecutar la misma tarea con cada estrategia y medir aspectos como el número de llamadas al modelo, los tokens utilizados y el tiempo de ejecución. También es necesario comprobar si la respuesta resuelve correctamente la solicitud. Las pruebas deben repetirse porque los modelos no siempre producen los mismos resultados. La clase destaca que una arquitectura más compleja no necesariamente es mejor: debe evaluarse según su precisión, coste, velocidad y facilidad de supervisión.

Supervisión humana y preparación para producción

En aplicaciones reales, el agente necesita límites y mecanismos para gestionar los casos que no puede resolver. Una opción es escalar la solicitud a una persona, por ejemplo, cuando faltan permisos, una herramienta falla o el problema supera las capacidades del sistema. También se muestra la utilidad de registrar trazas y métricas con herramientas como MLflow, para analizar las ejecuciones y detectar errores. La autonomía debe ajustarse al riesgo de cada tarea, manteniendo la posibilidad de intervenir cuando sea necesario.

Este documento ha sido generado con inteligencia artificial.